

Abgabe und Beurteilungskriterien

Jede Hausübung wird mit 100% bewertet, wobei diese 5% der Gesamtbewertung der ILV ausmachen.

Geben Sie die Hausübung bis zum 27.09.2026 um 23:59 via Moodle ab.

Abzugeben ist ein ZIP-File, welches das Java-Projekt + ein PDF mit der Dokumentation beinhaltet.

Benennung: **nachname_uebung1_bswe_algo_ws2026.zip**

Kann ein Java-Projekt nicht kompiliert werden, gibt es dafür Punkteabzug.

Bitte verwenden Sie das bereitgestellte Template.

Bewertet wird vorrangig die korrekte Funktionalität. Sind laut Angabe bestimmte Vorgaben zu erfüllen (beispielsweise das Verwenden einer bestimmten Kontrollstruktur), gibt es Punkteabzug, wenn diese nicht erfüllt werden.

Sollten Fragen oder Unklarheiten zu der Hausübung auftauchen, bitte ein E-Mail an patrizia.sailer@hochschule-burgenland.at oder sebastian.saip@hochschule-burgenland.at senden.

Die Bewertung wird zeitnah nach der Deadline erfolgen.

Aufgabenstellung

Aufgabenstellung: 9x9 Sudoku-Solver in Java

Ziel:

Implementieren Sie einen Sudoku-Solver in Java, der ein unvollständiges 9x9 Sudoku-Rätsel löst. Der Solver sollte das Rätsel so ausfüllen, dass alle Sudoku-Regeln eingehalten werden:

1. Jede Zeile muss die Zahlen 1 bis 9 ohne Wiederholungen enthalten.
2. Jede Spalte muss die Zahlen 1 bis 9 ohne Wiederholungen enthalten.
3. Jedes der neun 3x3 Unterquadrate muss die Zahlen 1 bis 9 ohne Wiederholungen enthalten.

Anforderungen:

1. **Eingabe:** Eine 9x9 Matrix (2-dimensionales Array) von Ganzzahlen, wobei leere/ungelöste Felder mit der Zahl 0 dargestellt werden. Die Eingabe der Felder soll über ein Menü möglich sein.

Es soll weiters möglich sein, eine csv-Datei mit dem Namen „input.csv“ einzulesen. Die Datei hat KEINE Header-Zeile. Werte werden durch einfache Komma (,) getrennt und werden NICHT in Hochkomma escaped.
Zeilen = Zeilen, die jeweiligen Werte darin = Spalten

Eine Beispiel-Zeile lautet somit:

1,2,3,0,7,0,8,5,9

Der Einstiegspunkt der Solver Klasse ist eine Methode mit folgender Signatur:

public static int[][] solve(int[][])

2. **Ausgabe:** Ein gelöstes 9x9 Sudoku (zusätzlich als System.out bzw. mittels Logger) oder eine Meldung, dass keine Lösung existiert.

Extrapunkte:

1. Erstellen Sie einen generischen Algorithmus, damit Sudokus mit unterschiedlichen Längen (4x4, 25x25, etc.) gelöst werden können. Es müssen nur Sudokus mit quadratischen Unterfeld-Größen berücksichtigt werden. Sonderfälle wie 12x12 (= 3x4 Unterfelder) müssen nicht gelöst werden.

Werden Werte größer 9 verwendet, sollen diese ebenfalls als Zahl eingegeben werden.

Für ein 25x25 Sudoku sollen pro Zeile/Spalte/Unterquadrat die Zahlen 1-25 vorkommen.

Das „input.csv“ enthält also bspw.: ... 3,20,5,0,12,25, ...

Aufgaben:

1. **Datenstruktur:** Wählen Sie eine geeignete Datenstruktur, um das Sudoku-Rätsel zu modellieren und effizient zu lösen. Sie können beispielsweise Arrays, Listen, Sets, Stacks, Queues oder eine Kombination davon verwenden. Begründen Sie Ihre Wahl der Datenstruktur.
2. **Algorithmus:** Implementieren Sie einen Algorithmus zur Lösung des Sudoku-Rätsels. Der Backtracking-Algorithmus ist eine naheliegende Wahl, aber Sie können auch andere Algorithmen oder Optimierungen verwenden, die Sie für geeignet halten.
3. **Validierung:** Implementieren Sie eine Funktion, die überprüft, ob ein bestimmtes Sudoku-Feld (Zeile, Spalte, Block) gültig ist, d.h., keine doppelten Zahlen enthält.
4. **Analyse der Algorithmuskomplexität:** Bewerten Sie die Zeit- und Platzkomplexität Ihres Algorithmus. Erklären Sie, wie effizient Ihr Algorithmus ist und welche Faktoren die Leistung beeinflussen könnten.
5. **Testfälle:** Erstellen Sie mehrere Testfälle, um die Korrektheit und Effizienz Ihres Sudoku-Solvers zu prüfen. Stellen Sie sicher, dass Ihr Solver sowohl lösbare als auch unlösbare Rätsel korrekt behandelt.
6. **Dokumentation:** Dokumentieren Sie Ihren Code klar und präzise. Erläutern Sie die gewählte Datenstruktur, den verwendeten Algorithmus und warum diese für die Lösung des Problems geeignet sind. Fügen Sie eine detaillierte Analyse der Algorithmuskomplexität bei.

Abgabe:

- **Code:** Geben Sie den vollständigen Quellcode Ihres Sudoku-Solvers in Java ab.
- **Dokumentation:** Fügen Sie eine schriftliche Erläuterung bei, die Ihre Wahl der Datenstruktur und des Algorithmus begründet, und eine Analyse der Algorithmuskomplexität (Zeit- und Platzkomplexität).
- **Testfälle:** Stellen Sie sicher, dass Ihre Lösung mit verschiedenen Testfällen getestet wird und fügen Sie die Ergebnisse bei.

Bewertungskriterien:

- **Korrektheit:** Die vorgegebene Schnittstelle muss implementiert werden. Der Solver muss korrekt funktionieren und alle Sudoku-Rätsel lösen können, die lösbar sind.
- **Effizienz:** Die gewählte Datenstruktur und der Algorithmus sollten effizient sein, besonders in Bezug auf Zeitkomplexität.
- **Codequalität:** Sauberer, gut dokumentierter und lesbarer Code. Keine unused methods etc.
- **Analyse der Algorithmuskomplexität:** Die Analyse der Zeit- und Platzkomplexität muss detailliert und nachvollziehbar sein.
- **Testabdeckung:** Umfangreiche und korrekte Testfälle, die alle Aspekte des Sudoku-Solvers abdecken.